

To: Charalampos Mainas (@cmainas) & Anastassios Nanos (@ananos) — Nubificus / urunc

15 May 2026

Project: CNCF · urunc · Improve lifecycle management of sandbox monitors

Upstream: urunc-dev/urunc#112

Repo: github.com/unchangedraman/proposal

This document is my proposal for the LFX 2026 Term 2 project on the urunc monitor-lifecycle issue. It covers the pre-application exercise you asked for in [comment 2 of #112](#) (reading urunc end-to-end and driving each monitor through its API), my background, the architectural question raised in that thread, my position on it, and a 12-week plan. The two prototype clients I wrote for the exercise are in the proposal repo under `prototypes/qmp-probe/` and `prototypes/firecracker-probe/`, with run transcripts.

1. About me

I am Raman Kapoor — a B.Tech (Information Technology) graduate from ABV-IIITM Gwalior (class of 2026), currently working at Juspay, India's largest payments orchestration platform, on low-level Linux infrastructure. My most recent landed piece there was leading the migration of our block-storage tier from CEPH to Lightbits NVMe/TCP: same workload at 48K TPS versus 37K TPS, on four nodes instead of forty, with active-active Barbican HA on a shared backend. The work I am on right now is the `wipe-disk` command for `nanna-cli`, our Rust CLI that drives bare-metal hosts in the datacenter through MAAS — secure-erase via NVMe Sanitize with an `hdparm --security-erase` fallback. Before Juspay, I wrote a 32-bit i386 kernel from scratch in C++ during undergrad — bootstrap, protected mode, PIC and ISR wiring, the GDT. That kernel project is what made me want to work on the userspace side of the same problem; urunc and this project are exactly that.

2. Pre-application exercise

The exercise required in #112 has three pieces: set up urunc and run a container; pick a supported monitor and drive a VM through its API; write up what was difficult and what surprised you. I have been running urunc locally since I filed [#389](#) a few weeks ago, so the first piece is already work I have shipped against the project. The two pieces below — driving QEMU through QMP, and driving Firecracker through its API — are the new work, plus a deliberate read of urunc's own source to understand the existing lifecycle before proposing a change to it.

2.1 Reading urunc's actual control flow

Until I read `cmd/urunc/create.go` end-to-end I had assumed urunc had a long-running process. It does not. Both `urunc create` and `urunc start` are transient — they exit after running their phase. The container's init process is QEMU or Firecracker itself, arrived at via a single `syscall.Exec` inside

the reexec process at `pkg/unikontainers/unikontainers.go:533`. Concretely, what I traced reading the source:

- `urunc create` (`cmd/urunc/create.go:90-280`) builds the `Unikontainer` state, sets up an init socket pair plus a log pipe for nsenter IPC, forks the reexec process via `createReexecCmd` at line 282, reads `{stage1_pid, stage2_pid}` JSON back from the init pipe, writes `state.json` with `containerPid = reexec_pid`, runs `CreateRuntime` OCI hooks, sends `AckReexec` to reexec on the init socket, then exits.
- The reexec process (`cmd/urunc/create.go:347-465`) runs inside the container's namespaces. It waits for `AckReexec`, runs `CreateContainer` hooks, creates a listener on `urunc.sock` (the `FromReexec` socket from `pkg/unikontainers/ipc.go:33`), and then **blocks on** `AwaitMsg(StartExecve)` at line 410. This is the gap the [slide deck](#) wants to use for parallel VMM init.
- `urunc start` (`cmd/urunc/start.go:38-115`) is a separate transient invocation. It sends `StartExecve` to reexec at line 83, waits for `StartSuccess` at line 102, sets the running state, runs `Poststart` hooks, and exits.
- Inside reexec, after `StartExecve` arrives, `Unikontainer.Exec` (`unikontainers.go:205-533`) runs `vmm.BuildExecCmd` at line 507, sends `StartSuccess` to urunc start at line 519, runs `vmm.PreExec` at line 525 (HVT seccomp etc.), and then calls `syscall.Exec(vmm.Path(), execCmd, env)` at line 533. The reexec process is replaced by QEMU/Firecracker. Same PID. That PID is the `containerPid` in `state.json` — which is what the shim and containerd track.

The two facts this exercise drove home, both of which matter for the architectural question in §4.2: there is no persistent urunc daemon on the host, and the container's init process is the monitor itself (not a supervisor). Any redesign has to respect both invariants or has to argue clearly for changing them.

2.2 QMP probe end-to-end

I wrote a small Go client that drives a real `qemu-system-x86_64 8.2.2` instance through its QMP Unix socket. The probe lives at `prototypes/qmp-probe/` in the repo; `run.sh` launches QEMU paused with `-S` and `-qmp unix:<path>,server,nowait`, then drives it through greeting, `qmp_capabilities`, `query-status` before and after `cont`, `query-cpus-fast`, `stop`, and `quit`. The full transcript runs in roughly 2 milliseconds:

```
[t+ 0.14ms] connected to /tmp/lfx-qmp-probe.sock
[t+ 0.53ms] <- greeting      {QMP: {capabilities: [oob], version: 8.2.2}}
[t+ 0.56ms] -> qmp_capabilities
[t+ 0.94ms] <- qmp_capabilities {return: {}}
[t+ 1.04ms] -> cont
[t+ 1.24ms] <- (async event) RESUME
[t+ 1.25ms] <- cont          {return: {}}
[t+ 1.48ms] -> query-cpus-fast
[t+ 1.71ms] <- query-cpus-fast [{thread-id, cpu-index, ...}, ...]
[t+ 1.88ms] <- (async event) STOP
[t+ 2.03ms] <- quit          {return: {}}
```

The piece of this that taught me something I would not have written into the proposal otherwise: **QMP interleaves async events with command replies on the same socket, in arrival order**. My first version of the client was naive — one read per command — and it worked through the greeting and the handshake and the first `query-status`. Then `cont` triggered a `RESUME` event that arrived before the `cont` reply, every subsequent read got off by one, and `query-cpus-fast`'s reply turned up where I was expecting the answer to `stop`. The fix is straightforward — type-dispatch every frame: `{return}` and `{error}` are replies to the most recent in-flight command, `{event, timestamp, data}` is async telemetry — but it is the kind of thing the docs do not flag and a code-from-spec approach would miss.

Five other concrete observations from the run, documented more fully in `prototypes/qmp-probe/README.md`:

- The greeting frame on connect is mandatory to consume before any client write.
- `qmp_capabilities` handshake is a gate — every other command is rejected until it has been sent.
- Reply payload schemas vary per command (`query-status` returns an object, `query-cpus-fast` returns an array, `quit` / `cont` / `stop` return `{}`). A single Go struct cannot statically type the Return field — `json.RawMessage` at the wire layer, parse per-command at the caller.
- `-qmp unix:<path>,server,nowait` is the correct flag combination. `nowait` matters specifically: it means QEMU initializes its devices and runs setup in parallel with whoever will eventually connect. That is the property the slide deck's parallel-init plan depends on.
- Latency is not the cost. Round-trip on a Unix socket is sub-millisecond. The cost of a socket-based control plane is in architecture and failure modes, not in latency.

2.3 Firecracker probe end-to-end

The companion exercise: a Go client that drives Firecracker v1.13.1 through its HTTP-over-Unix-socket API end-to-end. `prototypes/firecracker-probe/` in the repo. The script starts `firecracker --api-sock /tmp/lfx-fc-probe.sock`, then drives the lifecycle: `GET /` for sanity, `PUT /machine-config` with 1 vCPU and 128 MiB, `PUT /boot-source` with kernel and cmdline, `PUT /drives/rootfs` with the Ubuntu 22.04 ext4, `PUT /actions {InstanceState}` to boot, `GET /` to verify the state transitioned to Running, `PATCH /vm {state: Paused/Resumed}` to exercise pause/resume, and `PUT /actions {SendCtrlAltDel}` for graceful shutdown.

The microVM booted Ubuntu 22.04, reached systemd, honored the Ctrl-Alt-Del, and Firecracker exited with `exit_code=0`. Wall time was around 820 ms in total; the single most expensive call was `PUT /actions {InstanceState}` at 16 ms — the actual KVM boot. Every config call was 0.3 to 0.5 ms.

Six findings worth surfacing here (full version in `prototypes/firecracker-probe/README.md`):

- **Firecracker has no command-line equivalent for guest config.** Unlike QEMU's `-kernel ... -drive ... -netdev ...`, FC v1 accepts only `--api-sock`, `--id`, `--config-file`, and `--no-api`. There is no exec-with-everything-on-the-cmdline path. "Background mode" is the only mode. Whatever abstraction this LFX project lands must reconcile this with QEMU's exec-mode default.
- **All state-changing endpoints return 204 No Content.** The Go client cannot expect a response body for any mutation; state has to be read back via `GET /`.
- **No async events on the API socket.** Unlike QMP. To detect guest exit, you poll, observe the process exit, or read stderr. urunc's `MonitorControl` abstraction cannot assume events on every

backend — either expose `Events() <-chan Event` that QMP fills and FC leaves empty, or hide the asymmetry behind a capability flag.

- **Pause/Resume lives on `PATCH /vm`, not `/actions`.** FC's REST shape is slightly more REST-ful than QMP's flat command list. urunc's interface unifies them; per-monitor implementations translate.
- **No `device_add` equivalent.** FC supports `PATCH /drives/{id}` to swap an existing drive's backing path and updating network rate-limiters, but does not let you attach a brand-new virtio device after boot. Validates the need for capability flags on the interface.
- **The 16 ms InstanceStart is the parallel-init target.** All the config `PUT` s can happen during the gap where reexec blocks on `AwaitMsg(StartExecve)` ; only the final `PUT /actions {InstanceStart}` needs to land after `urunc start` . The QEMU equivalent is `-S` at spawn plus a QMP `cont` at start.

3. Background

3.1 Namma Yatri — open-source code I have shipped to production

Between November 2025 and January 2026 I worked as an open-source backend engineer on [Namma Yatri](#), the open-source ride-hailing platform that runs in production across India. Three merged PRs in the org:

- `feat nammayatri/nammayatri#13442` — Special-zone driver-payout scheduling system. Around 10,000 rides per day; driver payout latency dropped from weekly cycles to T+2 hours.
- `feat nammayatri/nammayatri#13452` — Exposed the resulting payout state to the driver-app frontend.
- `feat nammayatri/shared-kernel#1108` — PaytmEDC payment-flow integration in the shared kernel. Several review rounds before merge.

The technically interesting one is #13442. Driver payouts for special-zone rides (airports, stations) had been a weekly batch. Moving them to T+2 hours per ride meant roughly 10,000 scheduled jobs in flight at any moment under load. The constraints made the design non-trivial: a payout that fires twice is real money lost, a payout that never fires is a real driver unpaid; the ride-booking hot path could not block on scheduling; retries had to be idempotent. The shape I landed on was a Redis sorted-set keyed on ride-id with fire-time as the score, per-ride distributed locks taken at fire time, and an idempotency token persisted before the actual payout call. Scheduling is O(1) on the hot path, the worker pool polls the sorted set, and the per-ride lock plus idempotency token together guarantee exactly-once under broker restart, app restart, and partial-failure scenarios.

I mention this in some detail because it is the closest thing I have done to what this LFX project will involve: working on a large unfamiliar codebase (Haskell, no prior production experience), designing the failure model with maintainers, and getting to merged three times in a row.

3.2 Juspay — Lightbits, identity service, namma-cli wipe-disk

The Lightbits migration mentioned in §1 was the headline piece, but the two pieces most relevant to this LFX project's shape are the identity service and the namma-cli wipe-disk work.

Identity and secrets on OpenStack + OpenBao. We replaced a heavy AWS KMS dependency with an in-house identity-and-secrets layer. The interesting design choices: identity is derived from the VM's network position (IMDS-attested), not from a long-lived token on disk; the root key uses Shamir secret-sharing, split into m-of-n shards across the broker cluster so no single broker holds the full root; keys live in mlock'd pages and the broker refuses to start if core dumps are not sanitized; brokers run a Raft cluster across availability zones, so the service tolerates N-1 broker failures without an outage. The VM-side SDK (`osvault` , Python) handles attestation, token refresh, leasing, and audit. The error-domain modelling I did there — token-not-yet-issued vs token-expired vs broker-quorum-lost vs network-partitioned — is the same shape of work I expect to do around socket-lifecycle edge cases for QMP and the Firecracker API.

namma-cli wipe-disk. My current Juspay work is on `namma-cli` , a Rust CLI that drives bare-metal hosts through MAAS. Hosts are described declaratively in site specs; `namma-cli` reconciles spec to actual state. The shape is similar to Cluster API scoped to bare metal. I am working on the `wipe-disk` command — secure-erase before reprovisioning, using NVMe Sanitize with an `hdparm --security-erase` fallback. The piece worth surfacing here is an engineering-discipline find. The original wipe-disk code matched the SSH-exec result with a broad `Err(_)` arm and treated *any* SSH error as the legitimate "host poweroff disconnect", returning `Ok(())` . In practice that silently masked auth failures, connection-refused, and command-errors before the wipe script ever ran — every one of them returning `Ok` from a destructive operation that had not actually executed. For wipe-disk specifically, that is the worst possible failure mode: silent false-success on the operation that nukes customer data. I caught this on a local end-to-end test and proposed a discrimination heuristic — if the remote command produced no stdout before the disconnect, treat as an error rather than as expected poweroff. This is the same kind of error-domain discipline the urunc monitor-control plane will need to distinguish "QMP socket closed because we asked for `quit` " from "QMP socket closed because the monitor died."

3.3 Other background

PostgreSQL on Kubernetes / OpenStack. A 5-layer nested virtualization stack on a single i9-14900 — KVM under OpenStack under Kubernetes under CloudNativePG under Postgres — built to validate the same shape Juspay's production stack runs at on instrumentable hardware. Inner-VM Postgres write-TPS climbed roughly 50% after NUMA-aware CPU pinning and 16 GB of hugepages (the buffer pool was thrashing the TLB at default page sizes). Cluster API for OpenStack plus CloudNativePG bring a HA cluster up in around 12 minutes; primary failover is automatic, around 7 seconds to write-ready, with synchronous WAL streaming guaranteeing RPO=0. Same shape of problem as urunc — API-driven lifecycle of stateful infrastructure on top of VMM-based isolation.

BREAK OS. Already mentioned in §1; the small i386 kernel I wrote during undergrad. The reason it is in the proposal: the 16550 UART at I/O port `0x3F8` that I programmed in BREAK OS is the same hardware register the QEMU-emulated UART triggers PIO exits on, which is what made the urunc #389 trace recognizable rather than archaeological.

3.4 urunc so far — issue #389

A few weeks ago I picked up [urunc-dev/urunc#389](#) — high host CPU caused by console output. A prior contributor had attributed the regression to `bufio.Scanner` in `cmd/urunc/log_forward.go` ; my own profiling showed that path is not hot during guest execution. That scanner only handles the parent-child

JSON log pipe during `urunc create` ; once `syscall.Exec` runs (the same line 533 from §2.1), the VMM manages stdio directly. The real cost lives one layer deeper.

I wrote a small KVM-exit and throughput sampler — `bench-console.sh` , which @ananos has since indicated they would like to fold into urunc CI as a regression check (#389 comment 17) — and traced the regression to per-byte PIO traps on the 16550 UART at port `0x3F8` . The `-serial stdio` baseline costs roughly 1.16 M PIO exits per 5-second window, with QEMU's `address_space_write` dominating the host flame graph. A simple `virtio-console` swap drops PIO exits to ~0 and lifts throughput from 2,016 to 73,378 lines/sec (about 36×), closing the runc-vs-urunc gap on this workload from ~407× to ~11×.

The thread has since covered: how to differentiate `virtio-console` from `serial` without breaking Linux kernels built without `CONFIG_VIRTIO_CONSOLE` — my first proposal there (a capability-intersection design borrowed from urunc's rootfs path) was constructively rejected by @cmainas for exactly that reason; the upcoming pivot from the `com.urunc.unikernel.cmdline` annotation to a `kernelArgs` -style annotation that lets users specify boot parameters at build time; and an invitation from @ananos to draft a blog post on the investigation. The feedback shape from that round is what I expect from this project's design-doc review, and I have re-set my priors accordingly.

4. The project — monitor lifecycle for urunc

4.1 The problem in container-runtime terms

Following your steer in [#112 comment 10](#) — frame this in container-runtime terms, not generic VMM terms — the gap is not "urunc cannot do hotplug" but "urunc cannot do most of what a container runtime needs to do *after* the container's init process starts." Once `syscall.Exec` hands off to the monitor at [unikontainers.go:533](#) , the only handle urunc retains is the PID. `urunc kill` sends a signal to it. `urunc delete` cleans up around it. There is no way for urunc to ask the monitor what state the guest is in, to gracefully stop it, to attach a console after the fact, to add a new block device for a volume that wasn't part of the spec, or to detect guest exit without polling the process.

This is fine for the simplest container-runtime contract (start, run, kill) but insufficient for everything modern container runtimes are expected to do: graceful pause/resume across the OCI lifecycle, hotplug for dynamic-volume scenarios, console attach for `kubectrl exec` -style ergonomics, event-driven state tracking for the shim. Both QEMU and Firecracker already expose the necessary mechanisms — QMP and the FC API socket respectively — and the prototypes in §2 confirm both work cleanly. The work this project does is connect urunc's runtime-side state machine to them.

There is a secondary benefit that the [slide deck](#) calls out specifically: the reexec process currently blocks on `AwaitMsg(StartExecve)` between the create and start phases. The slides target "30-40 ms faster" boot by using that gap for parallel VMM initialization — spawn the monitor in background mode during the gap, send the final "start guest" command once `urunc start` arrives. The FC probe shows the actual KVM boot is around 16 ms; everything else (machine-config, boot-source, drives) is sub-millisecond and can happen during the gap.

4.2 The architectural question — and my position

@AyushSriv06 raised the central question in [#112 comment 7](#): preserve the current model where reexec eventually `execve`s into the monitor and persist the API socket for later urunc commands to reconnect to, or introduce a long-running supervisor process that owns the monitor and the persistent socket connection. You declined to pick for the applicants. I will pick for myself.

I would take the first approach. The container's init process stays the monitor itself; `state.json::containerPid` stays the monitor PID; no supervisor sits between the runtime and the container process. Concretely the new flow looks like this:

- During `urunc create`, the reexec process — after `CreateContainer` hooks but *before* blocking on `AwaitMsg(StartExecve)` — spawns the monitor in background mode as a child process via `fork+exec` (not `execve`). For QEMU that means `-S -qmp unix:<path>,server,nowait`; for Firecracker it means just `--api-sock <path>`. The monitor is its own process from this moment; the socket is open; `state.json::containerPid` is updated to the monitor's PID.
- Reexec then resumes its existing wait on `StartExecve`. While it waits, the monitor is in parallel initializing devices, mapping kernel images, opening drives — the work that previously had to happen serially after `StartExecve`.
- When `StartExecve` arrives, reexec connects to the monitor's socket and sends the "start guest" command — `cont` for QEMU (which is paired with the `-S` at spawn), `PUT /actions {InstanceStart}` for Firecracker. It sends `StartSuccess` back to urunc start. Then reexec exits cleanly. It does not `execve` into anything.
- For lifecycle operations after start — `urunc kill`, a new `urunc monitor query-status`, the shim asking for graceful shutdown — each command opens a fresh connection to the persisted socket, sends the operation, closes. The QMP probe shows round-trips are sub-millisecond on a Unix socket; the per-operation reconnect cost is not a concern. The fallback for sockets that have died is signal-to-PID, exactly as today.

The reason I prefer this over a supervisor process comes down to four points. First, OCI compatibility: container runtimes and the shim already expect the container's init process to be the monitor itself, not a wrapping supervisor. Changing that semantic is a bigger and less reversible decision than it looks. Second, failure-mode simplicity: with the supervisor model, a dead monitor with a live supervisor is a new zombie state to model; with the persisted-socket model, a dead monitor means a dead PID and a dead socket and urunc knows. Third, snapshot and restore (if either ever lands in urunc) cares about the monitor's state; the supervisor is incidental. Fourth, the persistent-connection benefit a supervisor would provide is small in practice: Unix-socket reconnect is sub-millisecond, and async-event subscription — the only thing that genuinely benefits from a long-lived connection — can be done by a separate event-sink process (or by the shim) without making the supervisor the container's init.

The trade-off I am explicitly accepting is that each lifecycle operation pays one socket-connect cost. Per the QMP probe, that is around 0.2 ms. I think that is the right trade-off for the OCI-compatibility win.

4.3 What changes in code

The work touches roughly six files. The list below is from the source-read in §2.1 and may shift during the design doc, but the shape is concrete enough to commit to:

- `pkg/unikontainers/hypervisors/qemu.go` — `BuildExecCmd` adds `-qmp unix:<sock>,server,nowait` and `-S` for paused start. Implements the new control interface for QMP: greeting + `qmp_capabilities` on connect, then per-method translation to `query-status`, `cont`, `stop`, `quit`, `device_add` / `device_del`, `chardev-add` / `chardev-remove`. The naive command list, prioritised from the prototype: `query-status` first (smallest, exercises every path), `quit` second (replaces the current `killProcess` in `Stop`), then pause/resume, then hotplug, then console attach, then introspection.
- `pkg/unikontainers/hypervisors/firecracker.go` — implements the same control interface backed by `firecracker-go-sdk`. The piece worth pulling out of the prototype: FC's lack of a CLI option for guest config means the entire boot sequence (machine-config, boot-source, drives, network) is API calls. The change is to do those calls incrementally during the create/start gap rather than as a single config-file load, and only invoke `PUT /actions {InstanceStart}` when `StartExecve` lands.
- `pkg/unikontainers/hypervisors/types.go` — extend the `VMM` interface (or, more likely, add a sibling `MonitorControl` interface — design decision, see §4.5) with the post-spawn operations. Capability flags on the interface for asymmetries like "QMP has events, FC does not" and "QMP supports device_add, FC does not," in the same shape urunc already uses for `SupportsSharedfs`.
- `pkg/unikontainers/unikontainers.go` — the load-bearing change. `Exec` (currently ending in `syscall.Exec` at line 533) splits into: spawn-monitor-in-background (called from `reexec` during the gap), wait-for-`StartExecve`, send-start-via-socket (replaces the `syscall.Exec`), send-`StartSuccess`. `Reexec` exits after the "send start" step. The IPC sequencing — making sure the monitor's socket is ready before `reexec` tries to talk to it — is the part @namansh70747 flagged in [#112 comment 5](#) as the trickier piece; I would handle it via the same retry-with-backoff pattern `sendIPCMessagesWithRetry` in `pkg/unikontainers/ipc.go` already uses for urunc's own sockets.
- `cmd/urunc/kill.go`, `cmd/urunc/delete.go` — update to attempt `MonitorControl.Shutdown()` first (QMP `quit`, FC `PATCH /vm + SendCtrlAltDel`), then fall back to `kill`-the-PID if the socket is broken or unresponsive. This is the same fallback discipline as the `namma-cli` wipe-disk error-handling work in §3.2.
- `pkg/unikontainers/ipc.go` — likely a small addition for any new state transitions (e.g. `MonitorReady` between background-spawn and `StartExecve`). The existing message constants may be enough; I will not know for sure until the design doc is written.

4.4 What I will not do, and what I will prove before each merge

Out of scope for this LFX, even if time allows: rewriting urunc's seccomp handling (`PreExec` for HVT), adding a third monitor (cloud-hypervisor, Dragonball, etc. — the interface should make adding one easier, but doing it is somebody else's first PR), introducing a heavy QMP or FC client library when both prototypes show the wire is small enough to hand-roll, or rewriting the existing `reexec/urunc-start` IPC channels. The work is layered on top, not a replacement.

The four things I commit to having proven before each respective merge: the QMP-controlled QEMU path can spawn paused, query state, gracefully shut down, and hotplug a virtio-net device end-to-end with the existing nginx-on-unikraft integration tests staying green; the Firecracker path can do the equivalent subset; the CLI-only fallback continues to work for HVT and any other monitor that does not expose a socket; an extended `bench-console.sh` measures the per-round-trip cost of monitor-control on both backends and lands in CI as a regression check.

4.5 Decisions the design doc will resolve

Four open questions where I have a working position but expect the design doc plus your review to settle the final shape. Listing them here so they are not buried in week-2 work:

- **QMP wire types** — hand-roll the small subset urunc actually uses (my current position, based on what the `qmp-probe` showed about reply schema variance), or adopt `github.com/digitalocean/go-qemu` or similar. I will benchmark dependency-impact in the design doc.
- **Interface placement** — extend the existing `VMM` interface, or add a sibling `MonitorControl` interface. My current position: sibling, because the lifetimes are different. `VMM` methods are valid from spawn; `MonitorControl` methods are valid from "after the socket handshake succeeds." Mixing them invites the caller to use a control method before the socket is ready.
- **Socket location and cleanup ownership** — per-container subdirectory under urunc's state dir (`/run/urunc/<id>/qmp.sock` or `fc.sock`) is the obvious choice. The question I do not yet have a good answer for is what happens if urunc crashes between socket-open and use. I would prefer to write that recovery policy into the design doc rather than discover it under test.
- **Opt-in vs default-on** — for the first cut, opt-in via an OCI annotation so existing users are not surprised. Once integration tests are green, flip to default-on for monitors that support it. This is the same pattern urunc already uses for shared-fs.

5. Plan — 12 weeks (8 Jun – 31 Aug 2026)

I plan to break the implementation phases into multiple small PRs rather than one large one, so review can happen in parallel with development. The design doc opens as a draft PR from week 1 and stays open through week 3 for asynchronous comments. The verification work in §2 reduces the risk on weeks 4–7 — I am not learning QMP or the FC API for the first time during the implementation.

WK	PHASE	CONCRETE DELIVERABLE
1–2	Onboarding + design doc, draft 1	Re-read <code>pkg/unikontainers/hypervisors/</code> with mentor context. Open a draft design-doc PR in <code>docs/</code> covering the <code>MonitorControl</code> interface, QMP and FC mapping, the reexec flow change, socket lifecycle, and resolving Decisions 1–4 from §4.5.
3	Design doc revisions, sign-off	Address review on the draft PR. Interface signature, dependency decision, and the opt-in/default-on policy are settled before any production code lands.
4–5	QEMU + QMP integration	Extend <code>qmp-probe</code> code into urunc's QEMU backend. MVP first (<code>query-status</code> + <code>quit</code> + <code>cont</code> — the smallest set that proves the integration), then full lifecycle (<code>stop</code> / <code>cont</code> for pause/resume, <code>device_add</code> /

WK	PHASE	CONCRETE DELIVERABLE
		<code>device_del</code> for virtio-net and virtio-blk hotplug). Unit tests for the wire codec.
6–7	Firecracker API + reexec rewiring	Same <code>MonitorControl</code> surface for FC, backed by <code>firecracker-go-sdk</code> . Land the reexec flow change: monitor spawned in background mode during the create/start gap, <code>syscall.Exec</code> path retained behind a feature flag.
8–9	Tests, CI, hotplug demo	Integration tests in the existing layout. Extend <code>bench-console.sh</code> to time the new round-trip costs on QMP and FC. End-to-end demo: start a unikraft-nginx sandbox, hotplug a second virtio-net device, verify connectivity inside the guest. Document the recipe.
10	Bug-fix + user-facing docs	Review-comment cleanup across all open PRs. User docs covering how to talk to a urunc sandbox's QMP socket and how to opt into the new control path.
11–12	Buffer + final clean-up + blog post	Reserved for slippage. If none, take a follow-up issue from the maintainers' backlog. Mentee blog post for the CNCF / Nubificus blog covering the redesign and the numbers.

5.1 Risk register

Twelve weeks is short. The risks I see and how I would handle each:

- **Design doc takes longer than two weeks.** The doc opens as a draft PR from week 1 so async review is possible. If week 3 still has unresolved structural questions, I deliberately pick the simpler of the two options to unblock the QEMU MVP and revisit the harder one mid-implementation, rather than blocking everything on consensus.
- **QMP socket-lifecycle edge cases are noisier than the design doc accounts for.** Built a small fault-injection harness during week 4 — kill the monitor at each step of the handshake, observe what reexec and urunc start each do. Edge cases surface there rather than in week-9 integration tests.
- **Firecracker SDK churn mid-project.** Pin `firecracker-go-sdk` in `go.mod` at week 6. Only upgrade if a fix we need lands upstream.
- **The reexec-flow change is more invasive than expected.** Feature-flag it from the start; the legacy `syscall.Exec` path stays the default until the new path has integration-test green in week 8. This is also the riskiest piece of the project — the IPC sequencing point @namansh70747 raised — so it gets the most buffer.
- **CI test-suite slowness blows up.** Budget half a week (part of week 8) to profile and trim. Precedent for that posture in §3.4 — using `perf` and KVM debugfs to find what is actually slow rather than guessing.

5.2 If the core scope lands ahead of schedule

Four follow-ups in order of priority, all explicitly out-of-scope unless time allows and you approve: wire the extended `bench-console.sh` into urunc CI with two-metric assertions (lines-per-sec floor, PIO-exits-per-window ceiling) so the regression #389 chased is caught automatically; implement `chardev-add` live console attach so `urunc attach <id>` works for an existing sandbox; subscribe to QMP

events (`SHUTDOWN` , `RESET` , `BLOCK_IO_ERROR`) as a long-running goroutine and surface them into urunc's state machine; experimentally wire a third backend (cloud-hypervisor) as proof the interface generalises.